1.0 LOCAL INTERCONNECT NETWORK (LIN)

1.1 Features

- Compliant to ISO 17987-3:2016 standard
- Compatible with LIN 1.3 and LIN 2.2 protocols defined in “LIN Specification Package 2.2A” by the LIN Consortium, 2010
- LIN protocol implementation at data link layer. Complete LIN Frame processing
- Transmission rates in the range from 1 kHz to 20 kHz
- AMBA APB host controller interface
- Configurable interrupt sources with interrupt enables for each error or status interrupt source
- Master enable bit for the LIN module. Master interrupt enable bit
- Full LIN message buffering of LIN Identifier and 8 data bytes
    - Automatic Frame Header reception
    - Protected Identifier buffering: The received ID is stored, enabling the host MCU to decide whether to process the incoming Frame Response or discard it
    - 8 data registers: Full message data buffering
    - Message parameters configuration: Specifying the number of data bytes, checksum type and inter-byte space insertion option
    - Control register: Enables host MCU to control the transmission of the Frame Response
    - Status report
    - Error report
- Automatic bit rate detection and frame synchronization without any prior programming of bit rate required
    - 1-20 kbps LIN bus speed operation
    - Automatic detection and processing of Break Field and Sync Field in the Frame Header
    - Break Field validation
    - Sync Field validation
    - Automatic bit rate detection by averaging 8 t_{bit} measurements
- Wake-up on lin_rx dominant level from transceiver
- Wake-up request support
    - Wake-up signal generation
    - Expiration times on wake-up signals
- Programmable bus idle time-out
    - Selectable bus idle time-out ranging from 4 to 10 seconds
    - Go-to-sleep function and notification
- Comprehensive error detection
    - Protected Identifier parity error
    - No-response time-out error
    - Framing error
    - Read-back error
    - Response too short error
    - Checksum error
    - Lost response error
- Collision detection feature and instant stop in case of collision
- One output irq signal to the system NVIC
- Support for classic checksum (LIN 1.3) and enhanced checksum (LIN 2.0)

1.2 Introduction

Local Interconnect Network (LIN) is a serial communication protocol focused on automotive applications. It is based on low speed universal asynchronous receiver transmitter (UART) networks. It provides a low-cost solution where the bandwidth and fault tolerance of a Controller Area Network (CAN) are not required. The LIN module is compliant with ISO 17987-3:2016 standard describing the protocol specification. The implementation takes care of the LIN protocol at data link layer, which requires minimal intervention from the host MCU. This relieves the burden on the MCU and leaves enough time for the MCU to complete other tasks.

1.3 Functional Description

The LIN block features dedicated hardware for automatic baudrate detection and synchronization. The Synchronizer detects the LIN Master's Break Field and Sync Field transmitted on the LIN bus in the Frame Header. Once the baudrate is detected and the bit time is extracted, the Frame Processor FSM takes care of the transmission flow control. Full LIN protocol is implemented at data link layer through dedicated hardware; The protected Identifier is processed, incoming transmissions are validated and stored, error detection and checksum check and generation are completed by the Asynchronous Serial Receiver and Transmitter block (ASRT). The module hosts an interrupt management system that will generate an IRQ to notify the host of any detected errors during transmission or any status updates. The LIN bus is also monitored to detect any time-out that would force the bus to sleep-mode, or to detect any wake-up requests transmitted on the bus by another node. The module also supports the ability to generate a wake-up request when the bus is put to sleep-mode with valid expiration periods of the transmitted wake-up signal. The host is able to access the register set through the AMBA APB interface. Note that the module is connected to the LIN transciever via GPIO pins. For more information, refer to the GPIO section. Figure 1-1 Shows the block diagram of the LIN module.

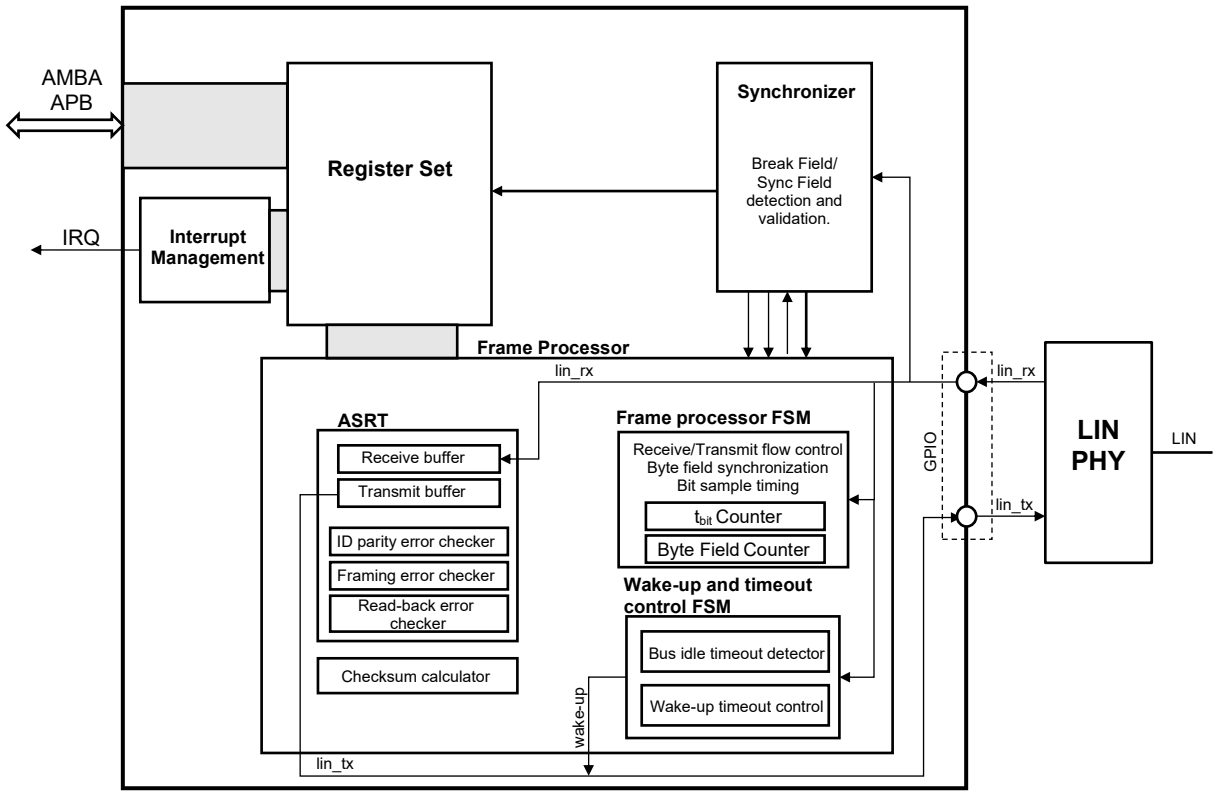


Figure 1-1: General block diagram of the LIN module

1.4 Register Set

The host can access the module's register set via the AMBA APB interface. Registers are word aligned. Reserved and unimplemented bits are read as zero.

The register map of the LIN module consists of 18 16-bit registers. Table 1 summarizes the register addresses, access types and reset values. The starting address of the LIN module is 0x4000C000.

Base Address: 0x4000C000

Table 1: Register map of the LIN module

Register Name	Access Type	Address Offset	Reset Value
lin_identifier	R	0x00	0x0000
lin_data1	R/W	0x04	0x0000
lin_data2	R/W	0x08	0x0000
lin_data3	R/W	0x0C	0x0000
lin_data4	R/W	0x10	0x0000
lin_data5	R/W	0x14	0x0000
lin_data6	R/W	0x18	0x0000
lin_data7	R/W	0x1C	0x0000
lin_data8	R/W	0x20	0x0000
lin_msgparams	R/W	0x24	0x0000
lin_control	R/W	0x28	0x0000
lin_status	R/1C	0x2C	0x0000
lin_errors	R/1C	0x30	0x0000
lin_tbit	R	0x34 or 0x38	0x0000
lin_csum	R	0x3C	0x0000
lin_config	R/W	0x40	0x000C
lin_status_inten	R/W	0x44	0x0037
lin_errors_inten	R/W	0x48	0x007F

lin_identifier

Reset value = 0x0000 **Address Offsets - 0x00**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	ID5	ID4	ID3	ID2	ID1	ID0
U	U	R	R	R	R	R	R

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ID<5:0>	5:0	LIN identifier extracted from the last received Frame Header. The identifier is stored after being validated with the parity check. An IRQ is triggered to the host MCU once a new identifier is stored to inform the MCU of a new valid identifier. This register is auto-cleared on the detection of a new valid Frame Header and a successful synchronization and bit rate extraction.

lin_datax

Reset value = 0x0000

Address Offset: **lin_data1: 0x04**
lin_data2: 0x08
lin_data3: 0x0C
lin_data4: 0x10
lin_data5: 0x14
lin_data6: 0x18
lin_data7: 0x1C
lin_data8: 0x20

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
DATA7	DATA6	DATA5	DATA4	DATA3	DATA2	DATA1	DATA0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
DATA<7:0>	7:0	LIN message payload. Incoming payload that has been verified by the LIN module will be stored in these registers for the host to read out. If an error is detected during the reception of a LIN message, the payload data will not be stored. Outgoing payload is written to these registers by the host before commencing a transmission.

lin_msgparams

Reset value = 0x0000 **Address Offset: 0x24**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	CSUM_TYPE	IB_SPACE	NBYTE2	NBYTES1	NBYTES0
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
NBYTES<2:0>	2:0	Number of bytes transmitted in the Frame Response of a LIN message(1). • During reception: This field is updated by the host after processing the received identifier to inform the LIN module of the number of bytes of data that are expected to be received. • During transmission: This field is updated by the host before initiating the transmission to inform the LIN module of the number of bytes to be transmitted. 000: 1 byte 001: 2 bytes 010: 3 bytes 011: 4 bytes 100: 5 bytes 101: 6 bytes 110: 7 bytes 111: 8 bytes
IB_SPACE	3	Inter-byte space. Option to insert one t_{bit}-long inter-byte spaces in the Frame Response when the LIN module is transmitting a message. 0: Inter-byte space is not inserted in the Frame Response 1: Inter-byte space is inserted in the Frame Response
CSUM_TYPE	4	Checksum type(1). Specifies the checksum type in the currently processed frame. • During reception: This field selects the checksum type to be used to evaluate received data bytes during reception. • During transmission: This field specifies the checksum type of the generated checksum byte in an outgoing LIN message. 0: Enhanced checksum (LIN 2.0) (default) 1: Classic checksum (LIN 1.3)
	Notes	(1) Diagnostic Frames with identifiers 60 (0x3C) or 61 (0x3D) always use the classic checksum and must always contain eight data bytes. The host must make sure to set these configurations accordingly and provide 8 bytes of data before initiating an outgoing transmission. Internal hardware will overwrite these configurations -only in this case- to the correct configurations if a wrong configuration is set to avoid any framing errors.

lin_control

Reset value = 0x0000 **Address Offset: 0x28**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	WU_REQ	RXTX_CTRL1	RXTX_CTRL0
U	U	U	U	U	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
RXTX_CTRL<1:0>	1:0	Receive and transmit control bits. This field commands the LIN module to perform an action. The host shall process the received identifier and configure this field to inform the LIN module of the desired action in the Frame Response. These bits will be automatically cleared after successfully processing of a Frame Reponse or upon the detection of a new Frame Header. 00: Default state. The received response will be discarded(1,4). (default) 01: Transmit the Frame Response(2). 10: Receive the Frame Response. 11: Discard the Frame Response(3).
WU_REQ	2	Wake-up request. If the LIN module is in sleep state (lin_status.sleep_mode = 1), setting this bit field to '1' will trigger a wake-up signal generation on the LIN bus. This bit will automatically be cleared once the bus is awake. Writing '1' to this bit when not in sleep state (lin_status.sleep_mode = 0) will have no effect. Reading this bit while the LIN module is operating (not in sleep state) will always return 0. 0: Wake-up signal not generated 1: Wake-up signal generated

Notes

(1) In the default state, the LIN slave will automatically receive the first byte of any Frame Response transmitted on the LIN bus until the host specifies the desired action.
(2) When selected, the LIN module will start transmitting the Frame Response one t_{bit} time after the reception of the frame ID and setting rxtx_ctrl = 01 (whichever occurs last).
(3) If the user application does not need to process the current Frame Response, it is recommended to select this option (rxtx_ctrl = 11) to explicitly command the LIN module to discard any incoming data on the LIN bus and wait for a new Frame Header.
(4) Could possibly cause a conditional lost response error. See section 1.7.1.7

lin_status

Reset value = 0x0000 **Address Offset: 0x2C**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RESP_ON	SERVICE_REQ	SLEEP_MODE	WU_DET	ERROR_FLAG	RESP_SENT	RESP_REC	HEADER_REC
R	R/W	R	R/1C	R/1C*	R/1C	R/1C	R/1C

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
HEADER_REC	0	Frame Header received. Indicates that a new Frame Header has been successfully received. The new identifier is stored in the lin_identifier register. The host MCU has to process the received identifier and configure the LIN module to take the appropriate action. In case the Frame Response has to be received or transmitted by the LIN module, the user application shall configure the lin_msgparams and lin_control registers accordingly before the transmission takes place. Writing '1' to this bit field clears it. 0: No Frame Headers received 1: Frame Header received
RESP_REC	1	Frame Response received. Indicates that a new Frame Response has been successfully received. Received data bytes are available to be read from the data buffer registers (lin_data1 to lin_data8). Writing '1' to this bit field clears it. 0: No Frame Responses received 1: Frame Response received
RESP_SENT	2	Frame Response sent. Indicates that the Frame Response has been successfully transmitted. Writing '1' to this bit field clears it. 0: Frame Response not transmitted 1: Frame Response transmitted
ERROR_FLAG	3	General error flag. This bit is a logical OR of the following error bits in the lin_errors register: ERROR_FLAG = ERR_LOSTRESP | ERR_PARITY | ERR_NORESP | ERR_FRAMING | ERR_READBACK | ERR_RESPSHORT | ERR_CHECKSUM * In order to clear this bit, one of two options are available: • Clearing all corresponding error bits in the lin_errors register. • Writing '1' to this bit field which will also clear all asserted errors in the lin_errors register. This is a shortcut path of clearing the lin_errors register. 0: No errors detected 1: General error flagged. Error detected

WU_DET	4	Wake-up detected. This bit is set when the LIN module is in sleep state and a bus wake-up signal is detected. Writing '1' to this bit field clears it. 0: No wake-up detected 1: Wake-up detected
SLEEP_MODE	5	The LIN module has entered sleep state. This bit is asserted either when: • The LIN Master sends a go-to-sleep master request frame. • The LIN module detects bus inactivity for the duration specified by BI_TO_SEL[2:0] bits in the lin_config register. This bit is automatically reset to '0' upon receiving a wake-up signal or a new LIN Frame Header. 0: LIN module is operating 1: LIN module is in sleep state
SERVICE_REQ	6	Service request. This bit is set to '1' when the LIN module needs the host MCU to process data or specify configuration or action to be taken for the current LIN frame. It also notifies the MCU in case of any bus events like the reception of a go-to-sleep command, or a bus error. If interrupts are enabled, the IRQ signal is triggered if this bit is set. If polling is prefered, this bit shall be polled by the user's application. This bit is a logical OR of the following status bits: SERVICE_REQ = GOTOSLEEP_INT(1) | WU_DET | ERROR_FLAG | RESP_SENT | RESP_REC | HEADER_REC Writing '1' to this bit field clears the gotosleep_int notification signal. To clear this bit, all relevant status and errors bits described in the equation above must be cleared. 0: No service requested 1: Service requested
RESP_ON	7	Frame Response transmission is on-going. This is a read-only bit that informs the host of any on-going transmissions. Any message-parameters related configurations should not be altered when this bit is set to '1'. 0: No transmissions on-going 1: Transmission on-going

Notes (1) Internal signal to notify the host that the LIN Bus has entered sleep mode. The sleep condition is monitored using the SLEEP_MODE bit in the same register.

lin_errors

Reset value = 0x0000 **Address Offset: 0x30**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RESP_ERR	ERR_LOSTRESP	ERR_PARITY	ERR_NORESP	ERR_FRAMING	ERR_READBACK	ERR_RESPSHORT	ERR_CSUM
R/1C*	R/1C(1)	R/1C(1)	R/1C(1)	R/1C(1)	R/1C(1)	R/1C(1)	R/1C(1)

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ERR_CSUM	0	Checksum error. This error is set if the LIN module receives a Frame Response where the received checksum byte does not match the calculated checksum(2). The user application must discard the received message in case of a checksum error. Writing '1' to this bit field clears it. 0: No checksum errors detected 1: Checksum error detected
ERR_RESPSHORT	1	Response-too-short error. This error is set if the LIN module is expected to receive a Frame Response but does not receive the message in full within the maximum response time-out of 128 t_{bit}. The LIN module will abort the reception of this frame and goes into idle state waiting for a new Frame Header. Writing '1' to this bit field clears it. 0: No response-too-short errors detected 1: Response-too-short error detected
ERR_READBACK	2	Read-back error. As defined per the ISO 17987-3:2016 spec, if the LIN module is driving the LIN bus to transmit a message by the lin_tx line, it also reads back the bus by the lin_rx line. This error is set if a mismatch occurs between what level is being transmitted and what level is being read back. The transmission will be aborted immediately in this case and the module goes into idle state and waits for a new Frame Header. Writing '1' to this bit field clears it. 0: No read-back errors detected 1: Read-back error detected
ERR_FRAMING	3	Framing error. As defined per the ISO 17987-3:2016 spec, each byte field must begin with a start bit '0', contain 8 bits of data and ends with a stop bit '1'. This error is set if an error occurs in either the start bit, stop bit or both. In this case, the LIN module aborts the on-going transmission and goes into idle state waiting for a new Frame Header. Writing '1' to this bit field clears it. 0: No framing errors detected 1: Framing error detected

ERR_ NORESP	4	No-response time-out error. As defined per the ISO 17987-3:2016 spec, the maximum length of a Frame Response is 130 t_{bit}(3). Time-out period is fixed to 130 t_{bit}. This error is set if this time-out period passes without receiving any Response. In this case, the frame will be dropped and the module goes into idle state, waiting for a new Frame Header. Writing '1' to this bit field clears it. 0: No no-response errors detected 1: No-response error detected
ERR_PARITY	5	Protected Identifier parity error. This error is set if an error is detected in the received parity bits of the Protected Identifier. In this case, the communication is aborted and the LIN module goes into idle state, waiting for a new Frame Header. Writing '1' to this bit field clears it. 0: No parity errors detected 1: Parity error detected
ERR_LOSTRESP	6	Lost response error. This error is set if the host MCU was too late(4) to define both message parameters (lin_msgparams register) and the desired action for the incoming Frame Response (lin_control register), from the LIN module, after processing the received identifier. In case of a lost response, communication is aborted and the LIN module goes into idle state, waiting for a new Frame Header. Writing '1' to this bit field or a successful reception of a response clears the flag. 0: No lost response errors detected 1: Lost response error detected
RESP_ERR	7	Response error. This bit is added to assist the higher stack level in reporting slave node errors (RESPONSE_ERROR) to the LIN bus master as defined per the ISO 17987-3:2016 spec(5). The RESPONSE_ERROR signal is used to report responses (data and checksum) transmitted or received by a slave node that contain an error. This bit is a logical OR of the following errors detected in the Frame Response: resp_err = err_framing | err_readback | err_respshort | err_checksum *This field is cleared in one of two cases: • Cleared automatically when a subsequent Frame Response is transmitted successfully. • Cleared by writing '1' to this bit field by the host. For Event-Triggered Frame Responses the host MCU shall ignore this bit and clear it.

Notes
(1) Detection of a new Frame Header also clears the bit.
(2) Caution should be taken as to which checksum type is being used. Checksum type is selected via the lin_msgparams register.
(3) $t_{Response\_nominal} = (8+1)*10*t_{bit} = 90\ t_{bit}$
$t_{Response\_max} = 1.4*t_{Response\_nominal} = 126\ t_{bit}$
(4) The error only occurs if the host selects the desired action as "receive response (lin_control.rxtx_ctrl = 10)" when the Frame Response has already started and the first data byte has been completely received including the stop bit.
(5) Proper status management and reporting to the cluster must be implemented at higher stack level in software. This bit is only a tool to assist the software.

lin_tbit

Reset value = 0x0000 **Address Offset: 0x34-0x38(1)**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	TBIT11	TBIT10	TBIT9	TBIT8
U	U	U	U	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
TBIT7	TBIT6	TBIT7	TBIT4	TBIT3	TBIT2	TBIT1	TBIT0
R	R	R	R	R	R	R	R

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
TBIT<11:0>	11:0	Auto-detected bit rate t_{bit}. This read-only register provides information about the current bit rate used in LIN communication. The 12-bit unsigned number provides the length of one t_{bit} in units of microseconds. The LIN module automatically synchronizes to the incoming Frame Header and extracts the bit rate. This register is just for providing the information to the host controller.

Notes (1) Reading any of these APB bus addresses will return the contents of this register.

lin_csum

Reset value = 0x0000 **Address Offset: 0x3C**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CSUM7	CSUM6	CSUM7	CSUM4	CSUM3	CSUM2	CSUM1	CSUM0
R	R	R	R	R	R	R	R

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
CSUM<7:0>	7:0	Checksum. This read-only register provides information about the received checksum or the transmitted checksum byte field: • The received checksum information becomes available to the host for reading when RESP_REC is set in the lin_status register. • The transmitted checksum information becomes available to the host for reading host when RESP_SENT is set in the lin_status register.

lin_conf3ig

Reset value = 0x000C **Address Offset: 0x40**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	BI_TO_SEL2	BI_TO_SEL1	BI_TO_SEL0	INT_EN	LIN_EN
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
LIN_EN	0	LIN master-enable bit. When disabled, the LIN module goes into initial state (Similar to POR). All active interrupts are de-asserted. The following registers are cleared: • Status reporting registers: lin_status and lin_errors • Data registers: lin_identifier and lin_datax • Configuration registers: lin_msgparams and lin_control • Informative registers: lin_tbit and lin_csum 0: LIN module disabled (default) 1: LIN module enabled
INT_EN	1	Interrupt master-enable bit. When enabled, each interrupt source will trigger an IRQ if its corresponding interrupt enable bit is enabled in the lin_status_inten and lin_errors_inten registers. If disabled, no IRQ will be triggered and polling must be used instead. 0: Interrupts disabled (default) 1: Interrupts enabled
BI_TO_SEL<2:0>	4:2	Bus idle time-out select bits. Programmable time-out ranging from 4 to 10 seconds as defined in the ISO 17987-3:2016 spec. The LIN module will automatically go to sleep state if the LIN bus is inactive for a period specified by these bits. 000: 4 seconds 001: 5 seconds 010: 6 seconds 011: 7 seconds (default) 100: 8 seconds 101: 9 seconds 11x: 10 seconds

lin_status_inten

Reset value = 0x0037 **Address Offset: 0x44**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	GOTO_SLEEP_EN	WU_DET_EN	-	RESP_SENT_EN	RESP_REC_EN	HEADER_REC_EN
U	U	R/W	R/W	U	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
HEADER_REC_EN	0	Interrupt enable for HEADER_REC status bit. When enabled, an interrupt request (IRQ) will be triggered if the HEADER_REC bit is asserted(1). The information will still be reported in the lin_status register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for header_rec 1: Interrupts are enabled for header_rec (default)
RESP_REC_EN	1	Interrupt enable for RESP_REC status bit. When enabled, an interrupt (IRQ) will be triggered if the RESP_REC bit is asserted(1). The information will still be reported in the lin_status register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for resp_rec 1: Interrupts are enabled for resp_rec (default)
RESP_SENT_EN	2	Interrupt enable for RESP_SENT status bit. When enabled, an interrupt (IRQ) will be triggered if the RESP_SENT bit is asserted(1). The information will still be reported in the lin_status register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for resp_sent 1: Interrupts are enabled for resp_sent (default)
WU_DET_EN	4	Interrupt enable for WU_DET status bit. When enabled, an interrupt (IRQ) will be triggered if the WU_DET bit is asserted(1). The information will still be reported in the lin_status register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for wu_det 1: Interrupts are enabled for wu_det (default)

GOTO_SLEEP_EN	5	Interrupt enable for go-to-sleep events. When enabled, an interrupt (IRQ) will be triggered if sleep event occurs either due to bus idle time-out or the reception of a go-to-sleep master request frame(1). The sleep information will still be reported in the lin_status register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for sleep events 1: Interrupts are enabled for sleep events (default)

Notes (1) Interrupts must be enabled through the master enable bit (INT_EN) in the lin_config register.

lin_errors_inten

Reset value = 0x007F **Address Offset: 0x48**

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit 7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	ERR_LOSTRESP_EN	ERR_PARITY_EN	ERR_NORESP_EN	ERR_FRAMING_EN	ERR_READBACK_EN	ERR_RESPSHORT_EN	ERR_CSUM_EN
U	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ERR_CSUM_EN	0	Interrupt enable for ERR_CSUM error bit. When enabled, an interrupt request (IRQ) will be triggered if the ERR_CSUM bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_csum 1: Interrupts are enabled for err_csum (default)
ERR_RESPSHORT_EN	1	Interrupt enable for ERR_RESPSHORT error bit. When enabled an interrupt (IRQ) will be triggered if the ERR_RESPSHORT bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_respshort 1: Interrupts are enabled for err_respshort (default)
ERR_READBACK_EN	2	Interrupt enable for ERR_READBACK error bit. When enabled an interrupt (IRQ) will be triggered if the ERR_READBACK bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_readback 1: Interrupts are enabled for err_readback (default)
ERR_FRAMING_EN	3	Interrupt enable for ERR_FRAMING error bit. When enabled an interrupt (IRQ) will be triggered if the ERR_FRAMING bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_framing 1: Interrupts are enabled for err_framing (default)

ERR_NORESP_EN	4	Interrupt enable for ERR_NORESP error bit. When enabled an interrupt (IRQ) will be triggered if the ERR_NORESP bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_noresp 1: Interrupts are enabled for err_noresp (default)
ERR_PARITY_EN	5	Interrupt enable for ERR_PARITY bit. When enabled an interrupt (IRQ) will be triggered if the ERR_PARITY bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_parity 1: Interrupts are enabled for err_parity (default)
ERR_LOSTRESP_EN	6	Interrupt enable for ERR_LOSTRESP error bit. When enabled an interrupt (IRQ) will be triggered if the ERR_LOSTRESP bit is asserted(1). The information will still be reported in the lin_errors register even when this bit is disabled. Only interrupt generation is affected by changing this field. 0: Interrupts are disabled for err_lostresp 1: Interrupts are enabled for err_lostresp (default)

Notes

1. Interrupts must be enabled through the master enable bit (INT_EN) in the lin_config register.

1.5 LIN protocol description

The LIN protocol is a single-master multi-slave inexpensive communication protocol aimed at reducing the cost in automotive applications in areas where the high bandwidths and strict fault tolerances of the Controller Area Network (CAN) is not needed. LIN communication does not support bus arbitration; Alternatively, the LIN bus Master organizes the communication to avoid bus collision by sending out Frame Headers to the entire cluster.

1.5.1 Message Frame

Communications are carried out in the form of Frames, where each Frame consists of a Frame Header sent solely by the LIN Master and a Frame Response sent either by the slave task in the LIN Master node or by any other slave node in the cluster. Figure 1-2 shows the LIN Frame format. The Frame Header is divided into Break Field, Sync Field and PID. The Frame Response carries the data bytes -payload- that are transmitted in a LIN message. The maximum number of bytes in a single transmission is 8 bytes of data. Following the last byte of data, a checksum byte is sent to ensure the validity of the transmitted data.

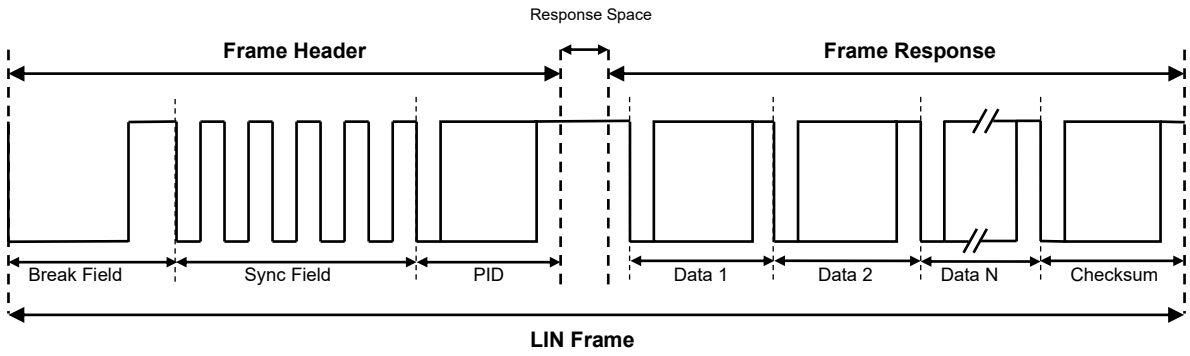


Figure 1-2: LIN Frame structure

1.5.1.1 Byte Field

All byte fields in a LIN Frame -with the exception of the Break Field- follow the structure shown in Figure 1-3. The least significant bit (LSb) of the data byte is sent first and the most significant bit (MSb) is sent last. The whole byte is encapsulated with a start bit, encoded as a bit with value zero (dominant), and a stop bit, encoded as a bit with value one (recessive).

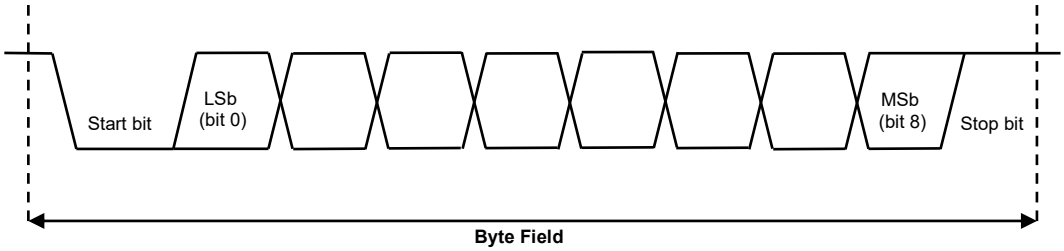


Figure 1-3: LIN byte field structure

1.5.1.2 Break Field

The Break Field is sent by the LIN Master to signal the beginning of a new Frame to the entire cluster. It is the only field that does not follow the structure of byte fields explained in section 1.5.1.1. The Break Field consists of at least 13 dominant bit times followed by a break delimiter of at least one recessive bit time. Figure 1-4 shows the structure of the Break Field.

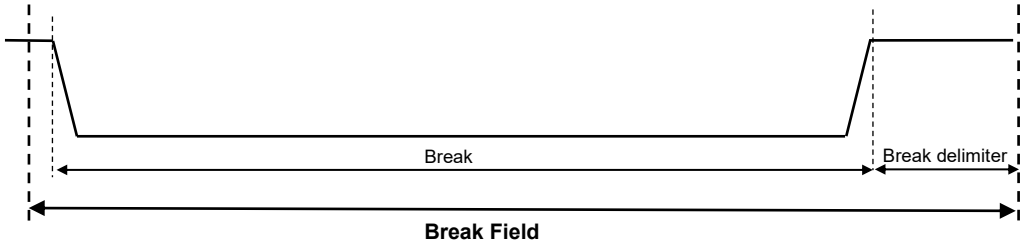


Figure 1-4: Break Field structure

1.5.1.3 Sync Field (0x55)

The Sync Field follows the Break Field in the Frame Header. Slave nodes synchronize to the Master's baudrate using this byte field. Upon detecting a Break/Sync Field sequence, a slave node shall abort any current transfers and be perpared to process the new incoming frame. Sync Field is a byte field with the data value 0x55 as shown in Figure 1-5.

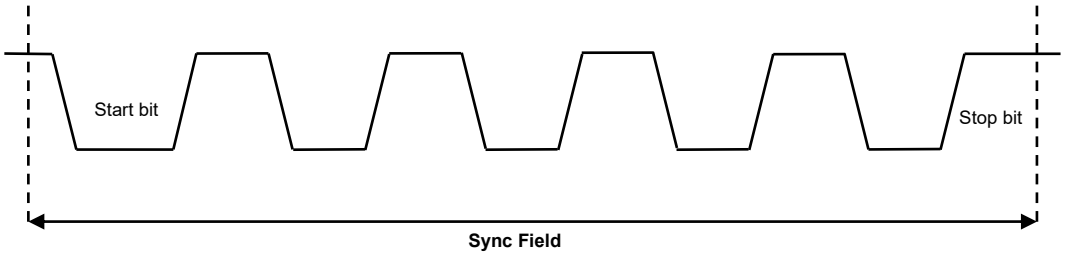


Figure 1-5: Sync Field structure

1.5.1.4 Protected Identifier (PID)

The PID field consists of two sub-fields: the frame identifier (bit5-bit0) and the parity bits (bit7-bit6) as shown in Figure 1-6.

1.5.1.4.1 Frame Identifier

Each Frame is associated with an identifier to inform the cluster what actions are required from which nodes. The six least significant bits of the PID are used as the identifier value. Meaning the identifier ranges from 0 to 63. Each node subscribes to a number of identifiers and the application layer is responsible of mapping each received idenifier with the desired action. Identifier values 0 (0x00) to 59 (0x3B) are used for signal carrying frames. Values 60 (0x3C) and 61 (0x3D) are used to carry diagnostic and node configuration data. The remaining values are reserved for future protocol enahancements.

1.5.1.4.2 Parity bits

The two most significant bits of the PID are used as parity bits (P_0 and P_1) to validate the sanity of the received identifier. If the received parity bits do not match the parity calculated from the received identifier. An error is flagged and the received identifier is discarded. The parity bits are calculated as follows:

$$P_0 = ID_0 \oplus ID_1 \oplus ID_2 \oplus ID_4 \quad (1)$$

$$P_1 = \neg\,(ID_1 \oplus ID_3 \oplus ID_4 \oplus ID_5) \quad (2)$$

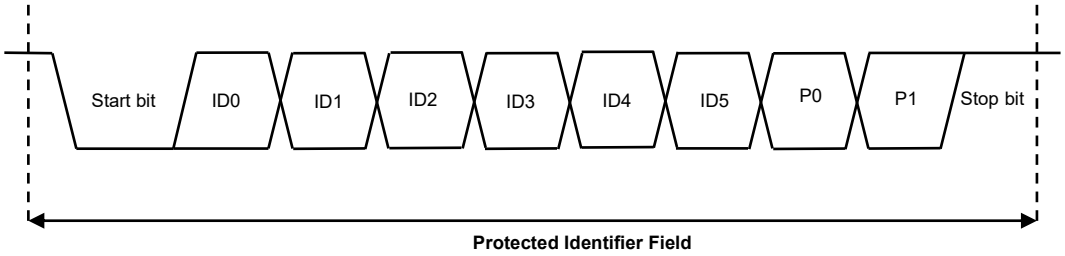


Figure 1-6: Protected Identifier (PID) Field structure

1.5.1.5 Data bytes

A Frame Response in an Unconditional Frame carries between one to eight data bytes. The number of data bytes is determined by the frame Identifier. Each slave node becomes aware of the expected number of bytes in the coming transmission after processing the identifier received in the Frame Header.

In Diagnostic Frames, the number of bytes is always set to 8 bytes of data in each message transmission.

A data byte follows the structure explained in section 1.5.1.1.

1.5.1.6 Checksum

The last field of the Frame Response and the LIN Frame is the checksum field. There are two checksum types:

- Classic checksum (LIN 1.3): The checksum byte contains the inverted eight-bit sum with carry over all Data bytes transmitted in the frame.
- Enhanced checksum (LIN 2.2): The checksum byte contains the inverted eight-bit sum with carry over all Data bytes and the Protected Identifier transmitted in the frame.

The used checksum type is determined by the LIN Master and defined per Frame Identifier. For Diagnostic Frames, only classic checksum is used.

If the received checksum does not match the checksum calculated from the received Data bytes, an error should be flagged and the received data shall be discarded.

The checksum byte follows the structure explained in section 1.5.1.1.

1.6 Operation

This section describes the behavior of the LIN module when a LIN Frame is being transmitted.

1.6.1 Frame Header Processing

The Frame Header is always transmitted by the LIN Master. The Break Field and Sync Field are used by slaves to synchronize to the transmitted message's bit rate. After synchronization, the PID Field is transmitted for slaves to process the frame Identifier and be ready with the appropriate action for the Frame Response.

1.6.1.1 Automatic synchronization and baud-rate detection

The synchronizer sub-module -see section 1.3- achieves three goals:

1. Continuously scanning the LIN bus to detect the beginning of a new LIN frame.
2. Validating the Break Field and Sync Field.
3. Automatically synchronizing to the incoming frame and extracting the transmission bit rate.

Unlike UART communication protocols, the LIN module automatically synchronizes and extracts the bit rate of the transmitted message. The module features dedicated hardware to detect and extract the bit rate from incoming transmissions. Meaning that the slave node is able to synchronize with the frequency at which the data is being transmitted on the LIN bus. The special timing properties of the Break Field and Sync Field in the Frame Header, defined in the ISO 17987-3:2016 spec, are used to extract the bit rate and validate the Break and Sync Fields without any intervention or prior-programming of baudrate from the host MCU.

The synchronizer sub-module scans the LIN bus continuously in the background, even if a Frame is being processed. Once a new Break/Sync Field sequence is detected and validated, the LIN module will drop the current Frame and start processing the new incoming Frame Header. This requirement is specified in the ISO 17987-3:2016 spec, which states that LIN slave nodes are to be ready to receive a new Frame Header at any given point even if an old Frame is being processed.

The module re-synchronizes to each and every received Frame Header. This means that the LIN module will be ready to receive multiple messages with different bit rates automatically without any programming or configuration.

Once a Break/Sync Field sequence is detected and validated. The bit rate is extracted and the value is stored in the lin_tbit register[1]. The synchronizer sub-module also flags a SYNC flag to the frame processor, to indicate that the synchronization process has been successfully completed and that the incoming PID Field needs to be processed.

If the incoming Frame Header fails the validation process, the LIN module will drop the frame and wait for a new Frame Header to be detected[2].

ISO 17987-3:2016 specifies LIN bit rate limits between 1-20 kbps. The LIN module supports bit rates ranging from 833 bps to 40 kbps with the onboard 40 MHz system clock.

[1] The obtained lin_tbit value is in 1 µs resolution. i.e, lin_tbit[11:0] = 50 means 50 µs t_{bit} detected – 20 kbps bitrate

[2] Deviation error in the sync field should not be outside the ISO 17987-3:2016 specification which allows a clock deviation of up to 14% between the slave and master oscillators.

1.6.1.2 PID Field processing

Once synchrnoized, the LIN module's frame processor becomes ready to receive and process the incoming PID Field. Firstly, the PID is validated using the received parity bits as explained in section 1.5.1.4.2.

If the parity check is passed successfully, the Identifier is stored in the lin_identifier register and the HEADER_REC bit in the lin_status register is asserted. An IRQ is triggered (if interrupts are enabled) to inform the host MCU of the reception of a new valid identifier. Based on the identifier value, the desired action for the Frame Response will vary[3]. The user application must process the received identifier and, in turn, configure the LIN module with the desired action.

If the parity check is not passed, the Identifier is discarded, the current frame gets dropped and the LIN module waits for a new Frame Header. In this case, the ERR_PARITY flag is set in the lin_errors register. This will trigger an IRQ (if interrupts are enabled) to inform the host MCU of the detected error.

For more information on interrupt generation, please refer to section 1.8.

1.6.2 Frame Response processing

The Frame Response carries up to eight data bytes and the checksum byte. This could be transmitted by the slave task in the LIN Master node or by any LIN slave. The LIN module can be configured to receive, transmit or ignore the transmitted message on the LIN bus. This section explains how to configure the module and how it operates in each of these scenarios.

1.6.2.1 Data reception

As shown in Figure 1-2, The Frame Response are separated by a Response Space, which is the time between the start bit of the first Data byte and the stop bit of the PID byte. This time is undefined and has to be a non-negative number according to the ISO 17987-3:2016 spec. This implies that the Response Space can theoretically be zero in some cases depending on the LIN Master. This might cause a problem in some cases:

Operating at the highest bitrate -20 kbps- each bit has a bit time of t_{bit} = 50µs. The PID stop bit is sampled in the middle of the t_{bit} period. This means that upon sampling the stop bit, framing errors will be checked and the MCU will be interrupted. The MCU will have to process the received Identifier and respond with the proper action for the Response Space within 25 µs, so that it could be ready to process the start bit of the first Data byte. The scenario is shown in Figure 1-7. This is a small window for the MCU which could be easily missed leading to missing the entire Response.

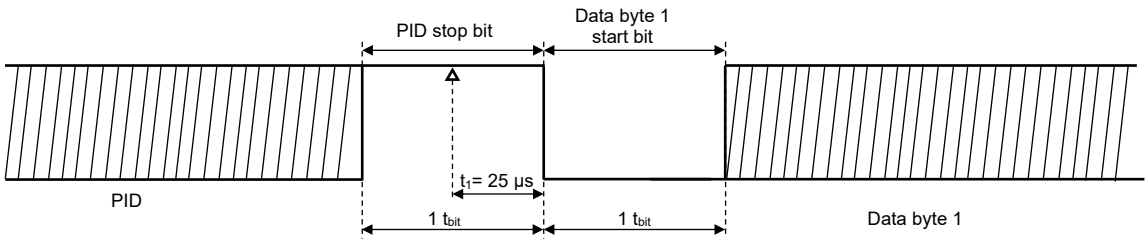
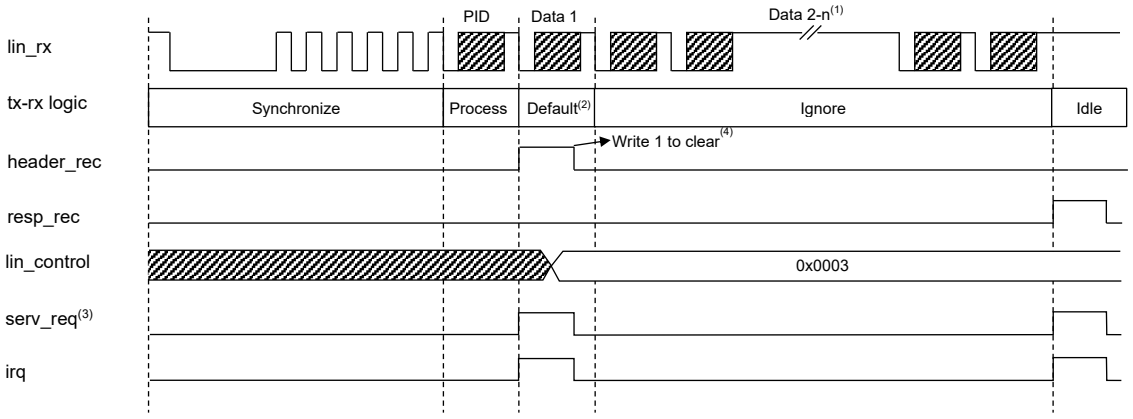


Figure 1-7: Zero Response Space scenario at 20 kbps (t_{bit}=50 µs)

[3] The interpertation of each Identifier value is described in the ldf file.

The LIN module is designed to handle this scenario efficiently. After receiving the Frame Header, the frame processor will enter the default state, waiting for the host MCU to specify the required action. However, if the module detects the start bit of a new Data byte (falling edge on the lin_rx line), it will automatically start processing the and receiving the first byte of data transmitted on the LIN bus. This behavior will relax the timing requirements forced on the MCU and will allow up to 500 µs (10 x t_{bit} running at 20 kbps) for the MCU to process the Identifier and configure the LIN module to the desired action. If the desired action is to:

- **Discard the Frame:** The host must write the lin_control register, specifying RXTX_CTRL to '11'. The LIN module will immediately discard the received byte and wait for the detection of a new Frame Header.

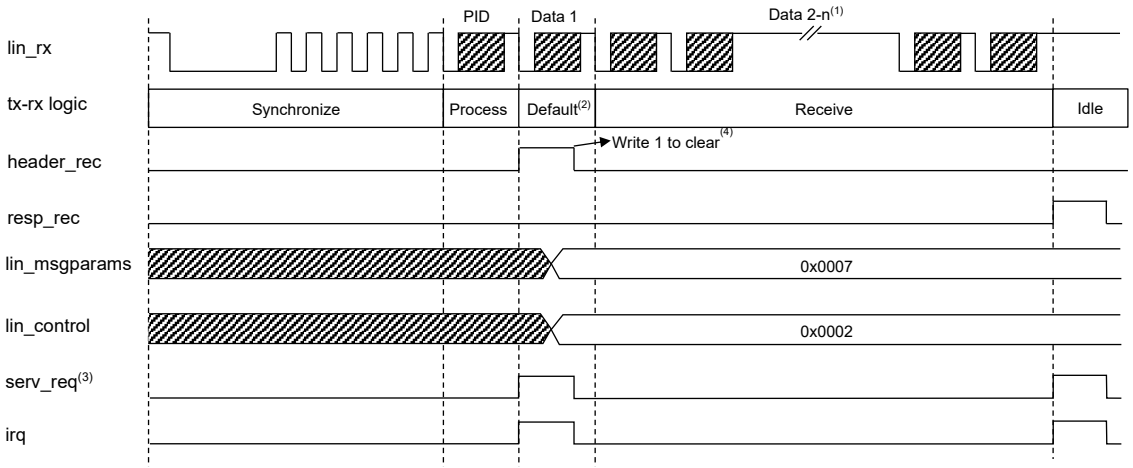


Notes:
(1) In this case, the LIN module is not subscribed to the transmitted frame. Thus, the transmitted data will be ignored.
(2) In this period, the LIN module is waiting for the host MCU to specify the desired action but will start receiving the first transmitted byte once a falling edge (start bit) is detected on the LIN bus. Once the host configures the lin_control register to ignore the transmission, the module will discard the received byte and ignore the rest of the transmission. It becomes idle and waits for a new Frame Header.
(3) Service request status bit (SERV_REQ) can be used for polling-based systems instead of interrupt requests.
(4) IRQ signals (or SERV_REQ status bit) are reset by writing '1' to clear the asserted flag in the lin_status register causing the interrupt.

Figure 1-8: LIN module behavior while discarding a transmitted frame

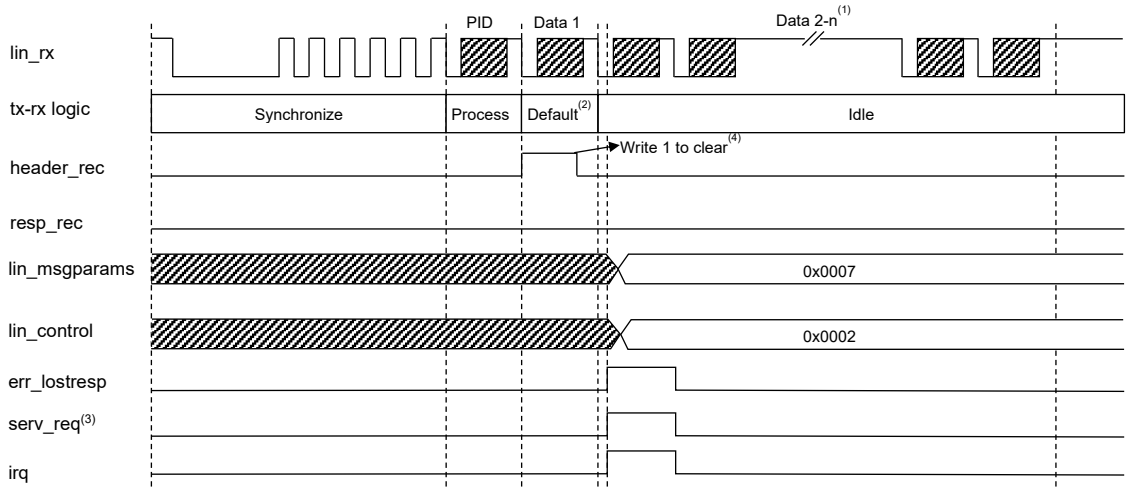
- **Receive the Frame:** The host must write the lin_msgparams configuration register and then write the lin_control register specifying RXTX_CTRL to '10'. The LIN module will continue receiving the complete Frame Response and inform the MCU when the reception is complete. If the host is too late to configure the LIN module, i.e. the first byte of the Frame Response has been received completely including the stop bit before the MCU configures the module, then the frame processor will set the ERR_LOSTRESP error flag in the lin_errors register and will discard the received byte[4]. The module will go into an idle state and wait for the detection of a new Frame Header. Figure 1-9 shows the behaviour of the LIN module in a successful reception of a Frame Response. Figure 1-10 shows the behaviour of the LIN module in a failed reception of a Frame Response due to a lost response error.

[4] This will only happen if the MCU specifies the desired action to receive mode.



Notes:
(1) Number of bytes (n) is set by the NBYTES[2:0] field in the lin_msgparams register. In this case, 8 bytes.
(2) In this period, the LIN module is waiting for the host MCU to specify the desired action but will start receiving the first transmitted byte once a falling edge (start bit) is detected on the LIN bus. Once the host configures the lin_control register to receive mode, it will continue receiving the rest of the transmission.
(3) Service request status bit (SERV_REQ) can be used for polling-based systems instead of interrupt requests.
(4) IRQ signals (or SERV_REQ status bit) are reset by writing '1' to clear the asserted flag in the lin_status register causing the interrupt.

Figure 1-9: LIN module behavior in a successful reception



Notes:
(1) Number of bytes (n) is set by the NBYTES[2:0] field in the lin_msgparams register. In this case, 8 bytes.
(2) In this period, the LIN module is waiting for the host MCU to specify the desired action but will start receiving the transmitted byte once a falling edge (start bit) is detected on the LIN bus. Since the host was too late to write the configuration registers (i.e. after the first byte was completely received including the stop bit), an ERR_LOSTRESP error is flagged and the frame processor goes into idle mode waiting for a new Frame Header. An IRQ is generated to inform the host of the occurring error.
(3) Service request status bit (SERV_REQ) can be used for polling-based systems instead of interrupt requests.
(4) IRQ signals (or SERV_REQ status bit) are reset by writing '1' to clear the asserted flag in the lin_status register causing the interrupt.

Figure 1-10: LIN module behavior in a response lost failure

Steps to properly configure the LIN module into receive mode:

1. LIN module will receive the Frame Header and validate the Identifier.
2. Once validated, the LIN module will flag the HEADER_REC status bit in the lin_status register and interrupt the MCU.
3. The MCU shall read the received identifier from the lin_identifier register.
4. The MCU shall process the identifier and depending on the application layer the required action is determined.
5. The MCU configures the LIN module to carry out the required action:
    a. Configure lin_msgparams and lin_control registers ('RXTX_CTRL = B'10') if the Response Frame is expected to be received.
    b. Configure lin_control register to ignore the transmission ('RXTX_CTRL = B'11') if no action is required for this Response Frame.
6. If required, the LIN module will continue receiving all data bytes, performing all data link layer functions.
7. Once all data is received successfully, the RESP_REC status bit is set in the lin_status register and the data bytes will be stored in the lin_datax registers and the LIN module interrupts[5] the MCU to indicate that the reception has been completed successfully.
8. MCU shall read the received response from the lin_datax registers.

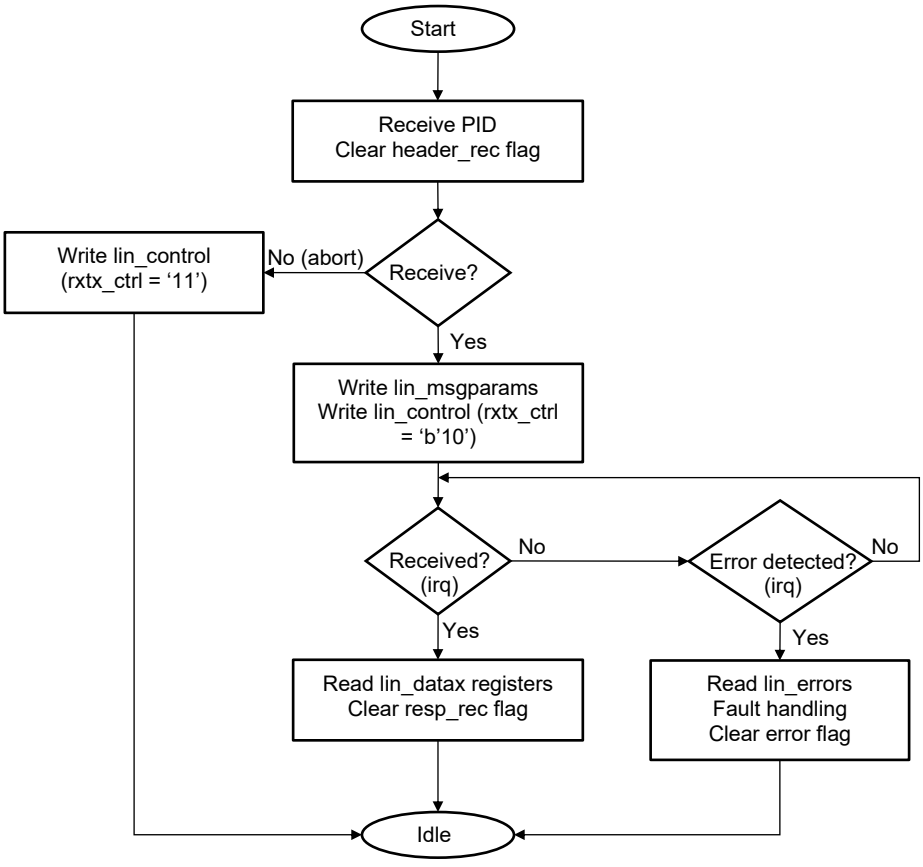
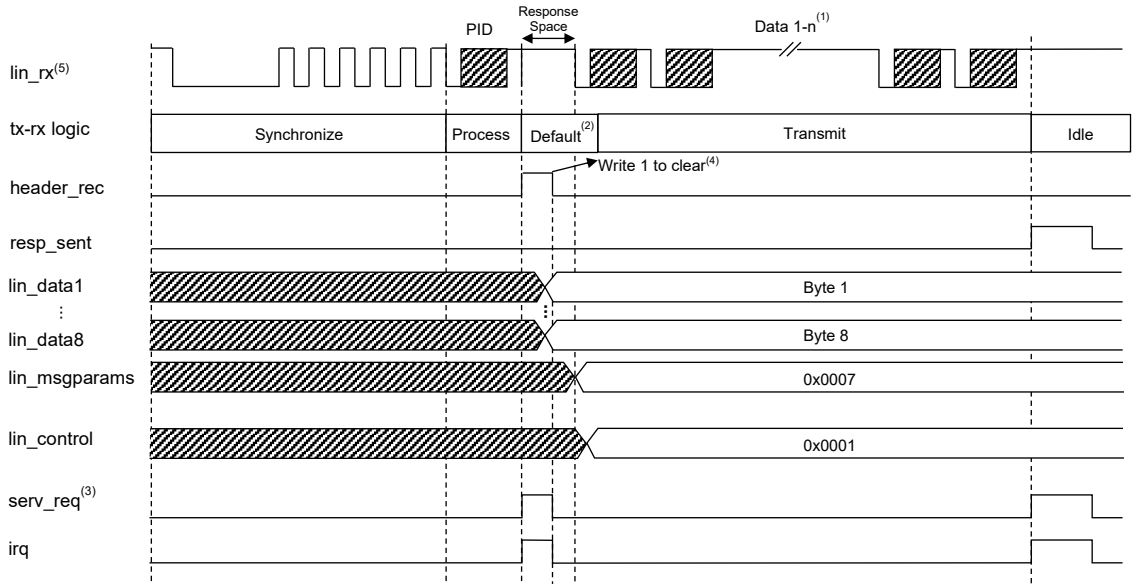


Figure 1-11: Flow chart of Frame Response reception

[5] Either interrupts or polling can be used.

1.6.2.2 Data transmission

As described in section 1.6.2.2, to accommodate for the zero Response Space condition, the LIN module enters a default state, upon receiving the PID byte, waiting for a configuration update from the host. If the received Identifier shows that the LIN slave is expected to transmit the Frame Response, ideally, nothing will be transmitted on the LIN bus. Therefore, there are no strict time restrictions on the host and once the host writes the data bytes to be transmitted (payload), updates the lin_msgparams register and configures the lin_control register to start transmission, the LIN module will commence transmitting the data unto the LIN bus.



Notes:
(1) Number of bytes (n) is set by the NBYTES[2:0] field in the lin_msgparams register. In this case, 8 bytes.
(2) In this period, the LIN module is waiting for the host MCU to specify the desired action but will start receiving any transmitted byte once a falling edge (start bit) is detected on the LIN bus. In this case, the LIN module is expected to be the publishing node. Therefore, nothing will be transmitted on the bus, i.e. A falling edge will never be detected.
(3) Service request status bit (SERV_REQ) can be used for polling-based systems instead of interrupt requests.
(4) IRQ signals (or SERV_REQ status bit) are reset by writing '1' to clear the asserted flag in the lin_status register causing the interrupt.
(5) Lin_rx is always monitoring the LIN bus. Even when the LIN module is transmitting the Frame Response.

Figure 1-12 LIN module behavior in a successful transmission

Steps to properly configure the LIN module into transmit mode:

1. LIN module will receive the Frame Header and validate the Identifier.
2. Once validated, the LIN module will flag the HEADER_REC status bit in the lin_status register and interrupt the MCU.
3. The MCU shall read the received identifier from the lin_identifier register.
4. The MCU shall process the identifier and depending on the application layer the required action is determined.
5. If the slave node is required to transmit the Frame Response:
    a. The host shall write the data bytes (payload) into the data registers (lin_datax).
    b. The host shall configure the lin_msgparams with the desired configuration.
    c. The host shall write the lin_control register (RXTX_CTRL = B'01) to start the transmission.
6. The LIN module will transmit all data bytes.
7. Once all data is transmitted successfully, the RESP_SENT status bit is set in the lin_status register and the LIN module interrupts [6] the MCU to indicate that the transmission has been completed successfully.

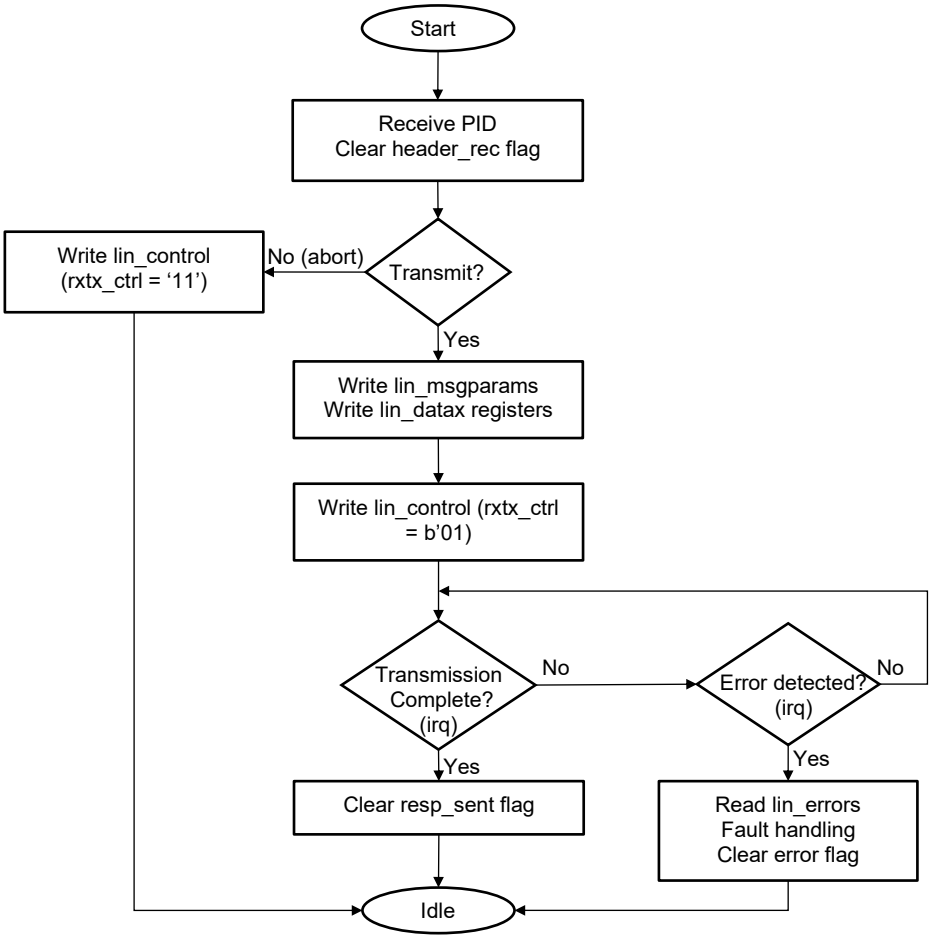


Figure 1-13: Flow chart of Frame Response transmission

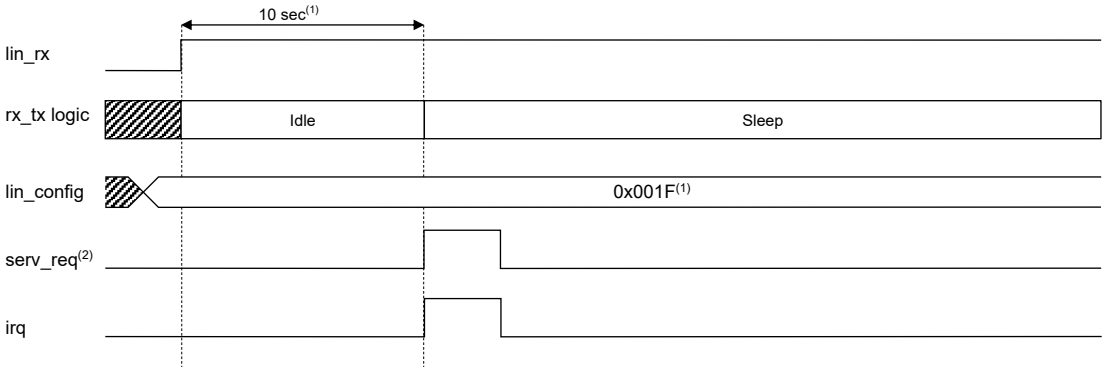
[6] Either interrupts or polling can be used.

1.6.3 Sleep Mode

The LIN module supports automatic bus-sleep mode[7] detection as defined in the ISO 17987-2:2016 specification. The module also features wake-up generation support and wake-up detection capabilities. This section explains the operation of the LIN module in each of these scenarios.

The bus sleep mode is entered on two ocassions:

- If the bus is idle, i.e. there is no traffic detected on the LIN bus, for a period from 4 to 10 seconds: The LIN module has dedicated hardware constantly monitoring the LIN bus checking for activity. A programmable timeout period is defined by the BI_TO_SEL<2:0> field in the lin_config register. If this defined period expires without detecting any activity on the LIN bus, an internal flag GOTOSLEEP_INT is asserted causing an interrupt signal to be generated (SERV_REQ in lin_status is set to 1)[8]. This will inform the host that the bus has gone into sleep mode. The read-only status bit SLEEP_MODE will automatically be asserted to indicate that the bus is asleep.



Notes:

(1) Timeout period is defined by the lin_config register.
(2) To reset the interrupt caused by the sleep event, write '1' to the SERV_REQ bit in the lin_status register.

Figure 1-14: Sleep event caused by bus inactivity timeout.

- If the LIN Master commands all slave nodes to enter bus sleep mode via a Diagnostic Frame (go-to-sleep command): As per ISO 17987-2:2016 specification, The LIN Master can send a go-to-sleep command in a special Master Request Frame (ID=0x3C) which forces all slave nodes to bus sleep state. This special Master Request Frame assigns the Node Address[9] (NAD) value to 0. If the LIN module receives a Master Request Frame (ID=0x3C) and the NAD value is 0, it will automatically assert the internal flag GOTOSLEEP_INT causing an interrupt signal to be generated (SERV_REQ in lin_status is set to 1)[7]. This will inform the host that the bus has gone into sleep mode. The read-only status bit SLEEP_MODE will automatically be asserted to indicate that the bus is asleep. Table 2 shows the go-to-sleep command transmitted by the LIN Master to instantly set all nodes into sleep mode.

Table 2: Go-to-sleep command

Master Request Frame	ID	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8
	0x3C	0x00	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF

[7] LIN bus sleep mode does not necessarily mean device sleep mode.
[8] To clear this interrupt, the host shall write '1' to SERV_REQ field in the lin_status register.
[9] Node Address (NAD) is the first byte of the Master Request Frame payload. Each slave has a unique NAD value. The LIN Master is able to address each slave using this byte.

1.6.4 Wake-up function

The LIN slave module supports wake-up detection as specified by the ISO 17987-2:2016 specification. The module also supports wake-up generation functionality, to wake the LIN bus if necessary. The A89201 features a device wake-up functionality through LIN. This section describes each of these functions in detail.

1.6.4.1 LIN wake-up

The LIN wake-up function is used when the LIN bus enters sleep mode (check section 1.6.3). A LIN wake-up sequence can be either transmitted by the LIN Master to wake the entire bus. Alternatively, a LIN wake-up sequence can be generated by a slave node to wake the bus and report the cause of this wake-up signal. It is the LIN Master's responsibility to investigate the source and reason behind a wake-up call signal when it is transmitted by a slave node.

The LIN slave module supports both wake-up signal detection and generation. Therefore, the LIN module is able to detect any wake-up signals transmitted by the LIN Master or any other Slave node, and is also able to generate a wake-up sequence to wake the LIN bus if necessary.

1.6.4.1.1 Wake up detection

The ISO 17987-2:2016 spec specifies a specific sequence that signals a wake-up command to all connected and powered nodes. The wake-up sequence is shown in Figure 1-15: Wake-up signalFigure 1-15. The wake-up sequence is started by forcing the bus to the dominant state for a period ranging from 250 µs to 5 ms and is valid with the return of the bus signal to the recessive state. A detection threshold of 150 µs is used by the slave nodes to detect the wake-up signal, which gives the uncalibrated slave nodes enough time to detect the wake-up signal.

If the LIN master transmits such a signal, it expects the Slave nodes to be ready for reception and transmission after 100 ms and will start sending new Frame Headers. If the node transmitting the wake-up signal is a Slave node, it shall be ready to receive and transmit frames immediately upon generating the wake-up signal.

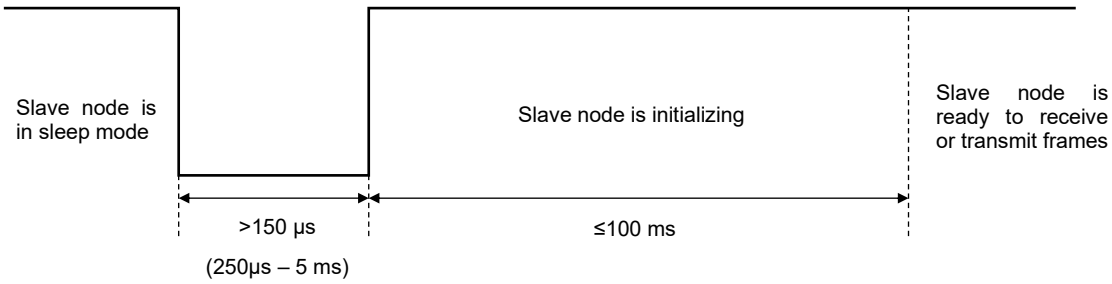


Figure 1-15: Wake-up signal

The LIN Slave module uses a detection threshold of 150 µs as specified by the ISO standard to detect the wake-up signal.

1.6.4.1.2 Wake-up generation

According to the ISO 17987:2-2016 standard, if a LIN Slave node does not receive a Break Field or a wake-up signal from another node after generating a wake-up signal within 150 ms to 250 ms from the wake-up signal, the Slave node issuing the initial wake-up signal shall re-transmit another wake-up signal. One block

of wake-up signals consists of three attempts to wake-up the LIN cluster separated by 150-250 ms waiting periods as shown in Figure 1-16.

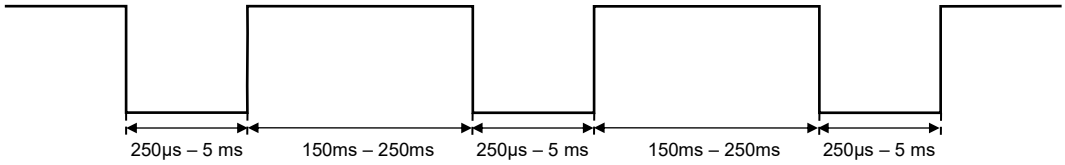


Figure 1-16: One block of wake-up signals

After three failing requests, i.e. one block, the node shall wait a minimum of 1.5 seconds before issuing another wake-up signal to give the cluster enough time to communicate in case there's a fault with the node issuing the wake-up signal. Figure 1-17 shows the wake-up signals over a long period of time.

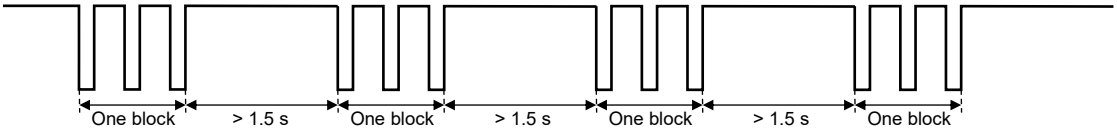


Figure 1-17: Wake-up signals over a long period of time

The LIN Slave module supports wake-up signal generation and complies with the ISO standard requirements. To initiate a wake-up sequence, the WU_REQ bit should be set to 1 in the lin_control register when the LIN cluser is in sleep mode[10]. The hardware will automatically keep track if any signal is received after transmitting a wake-up signal and will re-transmit the wake-up signal in compliance with the sequence explained in Figure 1-16 and Figure 1-17 if needed. Table 3 shows the timing values used in the LIN Slave module for each of the mentioned properties of a LIN wake-up signal.

Table 3: Timing properties of LIN Slave module's wake-up

Property	ISO 17987:2-2016 Specification	Allegro LIN Slave module
Wake-up detection threshold (Figure 1-15)	150 µs	150 µs
Wake-up pulse generation (Figure 1-15)	250 µs – 5 ms	512 µs
Wake-up re-try timeout within one block (Figure 1-16)	150 ms – 250 ms	200 ms
Wake-up retry timeout between blocks (Figure 1-17)	> 1.5 s	1984 ms (~ 2 s)

1.6.4.2 Device wake-up through LIN

The A89201 supports device wake-up through LIN. If the device is in sleep mode, the user may set the device to wake-up into awake mode upon the reception of a LIN awake event.

The device must be configured to use LIN as a wake-up source. To do so the following configurations must be set:

- The DWL bit in the GDU Config register[11] must be set to 0 to enable the wake-up through LIN feature.
- The OPM bit in the GDU Config register must be set to 0 to set the LIN pin to operate in LIN mode.

[10] The read-only bit SLEEP_MODE in the lin_status register indicates whether the bus is active or in sleep mode.
[11] For more information on the GDU configuration registers, refer to the GDU section.

After enabling LIN wake-up function, if the device is in sleep mode, a LIN awake event will wake the device up and the device becomes active. A LIN awake event is similar to the LIN wake-up signal described in section 1.6.4.1.1. The LIN awake event will be triggered if the LIN pin becomes dominant and remains dominant for at least 150 µs, and then becomes recessive. The event will be indicated at the rising edge when the LIN pin becomes recessive. Figure 1-18 shows a LIN awake event.

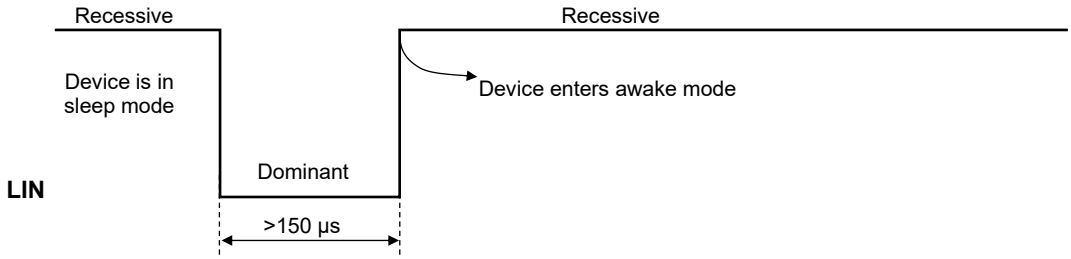


Figure 1-18: LIN awake event

1.7 Error management

This section describes possible LIN errors and how the LIN module handles them. The LIN module provides a comprehensive feedback of any errors that might occur in a LIN transmission. Errors may or may not generate an interrupt based on the user's preference. The option to enable or disable interrupt generation on the occurrence of any LIN errors is configurable through the lin_errors_inten register.

1.7.1 Error types

The following errors may occur in a LIN transmission:

1. Framing error
2. PID parity error
3. Response time-out (No-response error)
4. Read-back error
5. Response error (Response was too short)
6. Checksum error

In addition to the mentioned errors, the LIN module's architecture requires another bit to monitor the case of a late configuration as described in section 1.6.2.1. This condition is referred to as lost-response error in this document. All error information is available in the lin_errors register.

1.7.1.1 Framing error

Each byte field (except Break Field) must begin with a start bit (logic 0) and end with a stop bit (logic 1). The start and stop bits encapsulate 8-bits of data as shown in Figure 1-3. If this structure is not followed, the frame processor FSM will abort the ongoing communication and set the ERR_FRAMING error bit in the lin_errors register.

1.7.1.2 PID parity error

The two most significant bits of the protected identifier are reserved for the parity bits. These bits are used to validate the received identifier. The frame processor FSM will compute the parity with the received identifier using the formulas given by equations 1 and 2 and compare the result with the received parity bits. If a mismatch between the received bits and the calculated values occurs, the communication is aborted and the ERR_PARITY bit is set in the lin_errors register.

1.7.1.3 Response time-out (No-response error)

If the received identifier indicates that the LIN Slave is a subscriber to the Frame Response (i.e, expects to receive the incoming response) and the time-out period expires before receiving any data, the frame processor will abort the communication and set the ERR_NORESP bit in the lin_errors register.

The time-out period is fixed to 130 t_{bit}, which is longer than the maximum possible length (126 t_{bit}) for a Frame Response with 8 data bytes and a checksum byte.

1.7.1.4 Read-back error

In order to prevent faulty nodes from blocking the LIN bus, any Slave node publishing a message via the lin_tx line, i.e. driving the LIN bus, it shall also read back what is being transmitted on the LIN bus via the lin_rx line. If a mismatch between what level is being driven and what level is read back is detected, the frame processor will abort communication immediately and set the ERR_READBACK bit in the lin_errors register to prevent the node from blocking the LIN bus.

1.7.1.5 Response error (response was too short)

This error is set if the LIN module is a subscriber to a message but does not receive all expected data bytes, i.e. one or more bytes of data were not received, within the maximum response time-out period of 128 t_{bit}. In this case, the frame processor will drop the faulty frame and the ERR_RESPSHORT bit in the lin_errors register is set.

1.7.1.6 Checksum error

The checksum byte is sent by the message publisher at the end of the Frame Response after all data bytes are transmitted. The LIN module calculates the checksum from the received data[12]. If the received checksum does not match the calculated checksum. The ERR_CSUM is bit is set in the lin_errors register. However, the user application must discard the received bytes.

1.7.1.7 Lost-response error

The LIN module enters a default mode after receiving a valid identifier, where the frame processor will receive one byte of data if any traffic is detected on the LIN bus. Meanwhile, the host must configure the LIN module to perform the necessary action after processing the received identifier. If the host does not configure the LIN module before the first byte is received (stop bit is detected), the LIN module will discard the received byte and the ERR_LOSTRESP bit is set in the lin_errors register. The communication is aborted and the Slave becomes idle waiting for a new Frame Header.

1.7.2 Response error notification

According to the ISO 17987-3:2016 specification, a Slave node shall keep track of any errors that occur in the Frame Response and report it back to the LIN Master as a signal in one of its unconditional frames. It is the user's application responsibility to keep track of errors and report back to the LIN cluster. However, a single bit is added to the lin_errors register to aid the software in reporting this error.

The bit RESP_ERR bit is a logical OR of any of the LIN errors that are caused by a faulty Frame Response:

$$resp\_err = err\_framing \mid err\_readback \mid err\_respshort \mid err\_checksum \qquad (3)$$

This bit is set when any of the errors described in equation 3 are set after an error is detected in a Frame Response that is either transmitted or received by the LIN Slave module. This bit shall be ignored and reset by the user application in case of an event-triggered frame is received.

The bit is meant to be a tool to help the user software report to the LIN cluster. However, complete status management and reporting to the LIN cluster should be done in the application layer. 333

[12] Checksum type is defined in the lin_msgparams register and is calculated accordingly.

1.7.3 Clearing error flags

As a rule of thumb, writing '1' to any bit-field in the lin_errors register will clear it. In addition, some bits are reset under certain events. Table 4 summarizes how each bit is cleared. Alternatively, a shortcut path can be used to clear all error bits at once (Except for the RESP_ERR bit); To clear all the error bits at once, write '1' to the ERROR_FLAG bit in the lin_status register. The shortcut will automatically clear all bits in the lin_errors register without the need to access the register, which can save some MCU cycles.

Disabling the LIN module via the master enable bit LIN_EN in the lin_config register will reset the lin_errors register.

Table 4: Reset conditions for each error bit

Bit field	Reset conditions
ERR_CSUM	1. Writing '1' in this bit position clears this bit. 2. Detection of a new Frame Header clears this bit.
ERR_RESPSHORT	
ERR_READBACK	
ERR_FRAMING	
ERR_NORESP	
ERR_PARITY	
ERR_LOSTRESP	1. Writing '1' in this bit position clears this bit. 2. Detection of a new Frame Header clears this bit. 3. Successful reception of a Frame Response clears this bit.
RESP_ERR	1. Writing '1' in this bit position clears this bit. 2. Cleared automatically on successfully transmitting a Frame Response.

1.8 Interrupt management

The LIN Slave module is designed to handle complete LIN Protocol at the data link layer. Hence, minimum effort is needed from the user application. However, the interrupt management system ensures that any event will be reported to the host controller.

The LIN Slave module is compatiable with IRQ-based systems and polling based systems. The user may configure the module to generate an IRQ signal to the host's NVIC in case of an event, or the user application may poll a single bit to detect any event.

1.8.1 Interrupt-generating events

The lin_status register gives a comprehensive feedback to the application layer of the state of the LIN module and the latest events. The LIN Slave module generates an interrupt (or sets a polling bit) in case of one of the following events:

1. **Header reception:** This event occurs when the LIN Slave module receives a valid identifier. Once the identifier is received, it is stored in the lin_identifier register and the HEADER_REC bit in the lin_status register is set. The application shall read the received identifier and configure the LIN module with the appropriate action needed after processing the identifier.
2. **Response reception:** This event signals the successful reception of a LIN message. When a successful Frame Response is received and the checksum is validated, the RESP_REC bit in the lin_status register is set. The application shall read the received data from the lin_datax registers.
3. **Successful transmission of a response:** This event signals a successful transmission of a LIN message. When the host configures the LIN module to transmit a message, the module will automatically transmit each byte without any further intervention from the application layer. Once the transmission is complete without any errors, the RESP_SENT bit in the lin_status register is set. This bit is set to notify the application layer that the transmission was completed successfully. No further action is required or needed from the application layer.
4. **Error occurrence:** This event signals the occurrence of one or more errors in the latest LIN transmission. If one or more error flags described in section 1.7.1 is set, the general error flag ERROR_FLAG in the lin_status register is set.
5. **Wake-up signal detection:** This event occurs when the LIN bus is in sleep mode, and a wake-up signal is detected on the LIN bus. The WU_DET bit in the lin_status register is set.
6. **Entering sleep mode:** When the LIN module detects a sleep condition, bus idle time-out or go-to-sleep command, the frame processor will enter sleep mode and the SLEEP_MODE bit in the lin_status register is set. This bit is a read-only bit that displays the state of the LIN bus, and does not cause an interrupt. The interrupt is generated by an internal signal instead[13].

1.8.2 IRQ-based systems

The LIN Slave module generates a single IRQ signal to the host's NVIC. The IRQ signal is generated in each of the events mentioned in section 1.8.1. The LIN Slave module provides great flexibility with interrupt generation; Each and every interrupt source can be enabled or disabled by the user to generate an interrupt. However, it is not recommonded to disable any of the interrupt sources to guarantee full and proper functionality of the module.

All interrupt sources will set their corresponding bits in the lin_status register. Each interrupt source also has a corresponding interrupt generation enable bit in the lin_status_inten register. Similarly, interrupt sources caused due to errors in a LIN transmission will set the general flag bit ERROR_FLAG in the lin_status regsiter. The specific error bit in the lin_errors register corresponding to the interrupt-causing error event will also be

[13] The reason behind this approach is to display the state of the LIN bus if it enters sleep mode without causing a loop of interrupts.

set simulataneously. Each error bit has a corresponding interrupt-generation enable bit in the lin_errors_inten register.

For an interrupt signal to be generated for any event, the corresponding interrupt-generation bit must be enabled (set to 1). On top of that, a master-enable bit for interrupt generation and the LIN master-enable bit must be set in the lin_config register. When INT_EN is disabled, this bit will disable all interrupt generations. When enabled, the interrupt is generated if the specific enable bit is enabled for this interrupt. When the LIN master enable switch LIN_EN is disabled, all interrupts will be de-asserted and lin_errors and lin_status registers will be reset.

If the interrupt for a specific event or interrupts in general are disabled, the event will still be reported in the lin_status and lin_errors registers. Disabling the enable bits only affects the interrupt generation.

Figure 1-19 shows the interrupt generation logic. The GOTO_SLEEP_INT signal is an internal signal that is set whenever a sleep condition is detected[13].

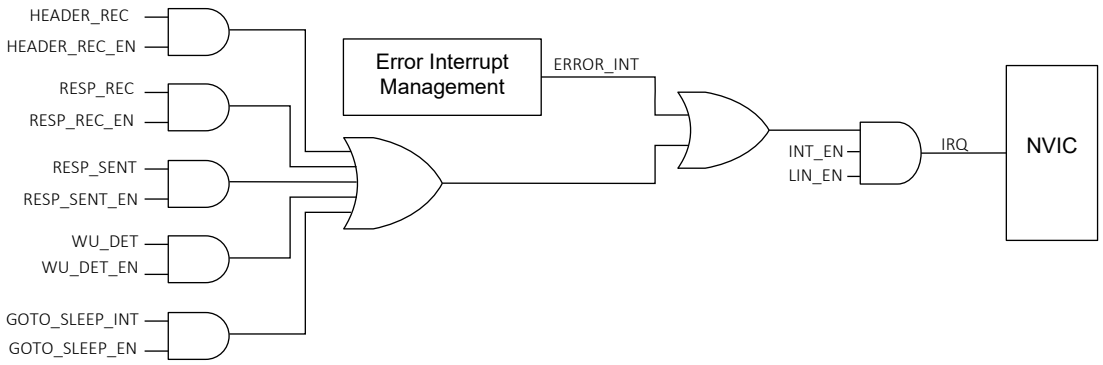


Figure 1-19: Interrupt generation logic

The ERROR_INT signal is generated from any error event with interrupt generation enabled. Figure 1-20 shows interrupt-generation management logic of LIN errors.

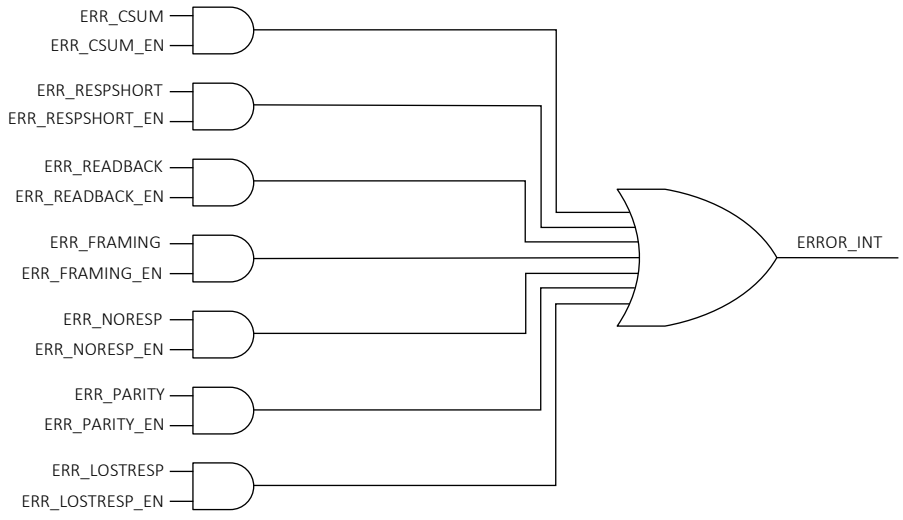


Figure 1-20: Error interrupt-generation management logic

1.8.3 Polling-based systems

The LIN Slave module provides support for polling-based systems. If it is prefered, all interrupt generation could be disabled and the host is able to poll a single bit to understand the current status of the module as well as the occurrence of any events.

The SERVICE_REQ bit in the lin_status register represents all the information about all of the events discussed in section 1.8.1. In other words, if any of the mentioned events take place, this bit will be set to report the change to the host controller. The user software shall poll this bit to monitor the status of the LIN module. After polling this bit, lin_status register shall be read for further investigation of the source of the generated flag. If the ERROR_FLAG bit in the lin_status register is set, the lin_errors register shall be read for more information.

The ERROR_FLAG bit is a logical OR gate of all the error bits in the lin_errors register. SERVICE_REQ bit is a logical OR of all the possible events reported in the lin_status regsiter and the ERROR_FLAG signal. These signals are not maskable and will always drive the SERVICE_REQ bit. Figure 1-21 shows the logic driving the SERVICE_REQ bit.

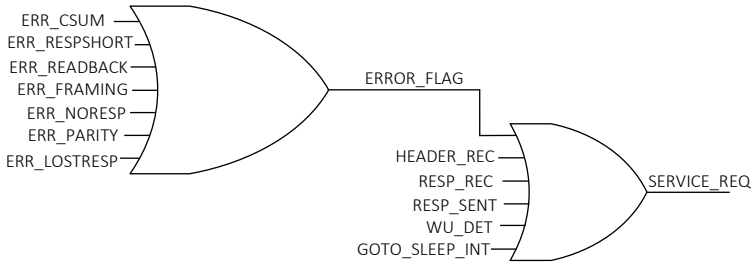


Figure 1-21: SERVICE_REQ bit logic

The ERROR_FLAG and SERVICE_REQ signals are always updated whether interrupts are enabled or disabled.

1.8.4 Resetting interrupts

To reset an interrupt or an asserted flag, the rule of thumb is to write '1' to that bit field position to clear it. However, there are some exceptions. Once the flag is reset, the IRQ signal will be de-asserted. Table 5 summarizes the effect of writing '1' to all flags in the lin_status register. Clearing the lin_errors register is discussed in section 1.7.3.

Disabling the LIN module will clear all bits in the lin_status and lin_errors registers. Any asserted IRQ signals will be de-asserted as well.

Table 5: Clearing the lin_status register bits

Bit field	Effect of writing '1' to this bit field
HEADER_REC	Writing '1' to this field will clear the flag.
RESP_REC	Writing '1' to this field will clear the flag.
RESP_SENT	Writing '1' to this field will clear the flag.
ERROR_FLAG	Writing '1' to this field will clear all the asserted flags in the lin_errors register.
WU_DET	Writing '1' to this field will clear the flag.
SERVICE_REQ	Writing '1'to this field will clear the internal signal GOTO_SLEEP_INT which indicates that a sleep event has been detected and the module is has entered sleep mode.

Revision Control:

Nov 20/2020	Initial Revision
Nov 11/2020	Updated Logo
Dec 02/2024	Added description for err_parity_en interrupt generation Corrected bit position for goto_sleep_en and wu_det_en